SIL765/7165 Systems & Networks Security

- Admin: Class III LT 3 Tu/We/Fri 10-11
- TA: Priyansh Singh, Jitesh Shamdasani, Abhiya Jose & Aquif
- Breakup (indicative)
  - Assignment + end sem project 32
  - Quiz 8
  - Midterm 25
  - Major 35

Reference Books/Material

- **Charlie Kaufman, Radia Perlman & Mike Speciner — *Network Security: Private Communication in a Public World***
  A classic, comprehensive reference on network security protocols and secure communications;
- **William Stallings — *Cryptography and Network Security: Principles and Practice***
  covering cryptographic techniques, network security principles, and standard protocols.
- **William Stallings — *Network Security Essentials: Applications and Standards***
  Focuses on fundamental network attacks/defenses and common standards in network security.
- **Ross J. Anderson — *Security Engineering: A Guide to Building Dependable Distributed Systems***
  Excellent for understanding security from an engineering and systems perspective.

Other Reference Material

- **Jonathan Katz & Yehuda Lindell — *Introduction to Modern Cryptography***
  Rigorous modern treatment of crypto foundations and proofs.
- **Jon "Smibbs" Erickson — Hacking: The Art of Exploitation**
  - Excellent for understanding low-level system vulnerabilities and exploit techniques.
- **Bill Blunden — The Rootkit Arsenal**
  - In-depth coverage of rootkits, kernel security, and advanced system-level malware techniques.

Key Papers

1. "The Protection of Information in Computer Systems" (Saltzer & Schroeder, 1975)

Introduced the foundational *design principles of secure systems* (least privilege, fail-safe defaults, open design, etc.).

2. "Reflections on Trusting Trust" (Ken Thompson, 1984)

Demonstrates compiler-level backdoors; a landmark on trust, supply-chain security, and the impossibility of fully verifying systems.

3. "End-to-End Arguments in System Design" (Saltzer, Reed, Clark, 1984)

Defines the famous *end-to-end principle*, core to modern network and distributed system design.

4. "A Logic of Authentication" (Burrows, Abadi, Needham — BAN Logic, 1989)

The BAN logic paper formalized reasoning about authentication protocols.
Essential reading for network security & protocol analysis.

5. "The Morris Worm" Analysis (Eugene Spafford, 1988)

The first major worm incident; analysis established terminology and fundamental approaches for understanding self-replicating malware.

Cryptography & Secure Protocols (key Papers)

6. “New Directions in Cryptography” (Diffie & Hellman, 1976)

Introduced *public-key cryptography* and *Diffie–Hellman key exchange*.

7. “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems” (Rivest, Shamir, Adleman — RSA, 1978)

Defines RSA. Cornerstone of secure communication and network protocols.

8. “A Secure and Efficient Network Authentication Service” (Kerberos — Steiner, Neuman, Schiller, 1988)

crucial influence on authentication, distributed systems, and real-world system security.

9. “How to Prove Yourself: Practical Identification Protocols” (Feige, Fiat, Shamir, 1987)

Pioneered zero-knowledge proofs; highly influential for secure authentication and modern cryptographic protocols.

10. “Authenticated Key Exchange and Secure Channels Based on Passwords” (Bellovin & Merritt, 1992 — EKE Protocols)

Key influence on modern password-based authenticated key exchange (PAKE) systems.

More Key Papers

- **"The Security Architecture for the Internet Protocol" (Kent & Atkinson, 1998 — IPsec)**
  - Formally defines IPsec; highly influential in VPNs, secure tunneling, and network-layer security.
- "The Byzantine Generals Problem" (Lamport, Shostak, Pease) — fault tolerance
- "Fuzz Revisited" (Miller et al.) — software robustness testing
- "Smashing the Stack for Fun and Profit" (Aleph One, 1996) — buffer overflow exploitation
- "Unifying Vulnerability Management" (various authors) — modern vuln classification approaches

"Security"

- Most of computer science is concerned with *achieving desired behavior*
- Security is concerned with preventing undesired behavior
  - Different way of thinking!
  - An enemy/opponent/hacker/adversary who is actively and maliciously trying to circumvent any protective measures you put in place

Security is interdisciplinary

- Draws on all areas of CS
  - Theory (especially *cryptography*)
  - Networking
  - Operating systems
  - Databases
  - AI/learning theory
  - Computer architecture/hardware
  - Programming languages/compilers
  - HCI, psychology

The computer security problem

- **Lots of buggy software**
- **Money can be made from finding and exploiting vulns.**

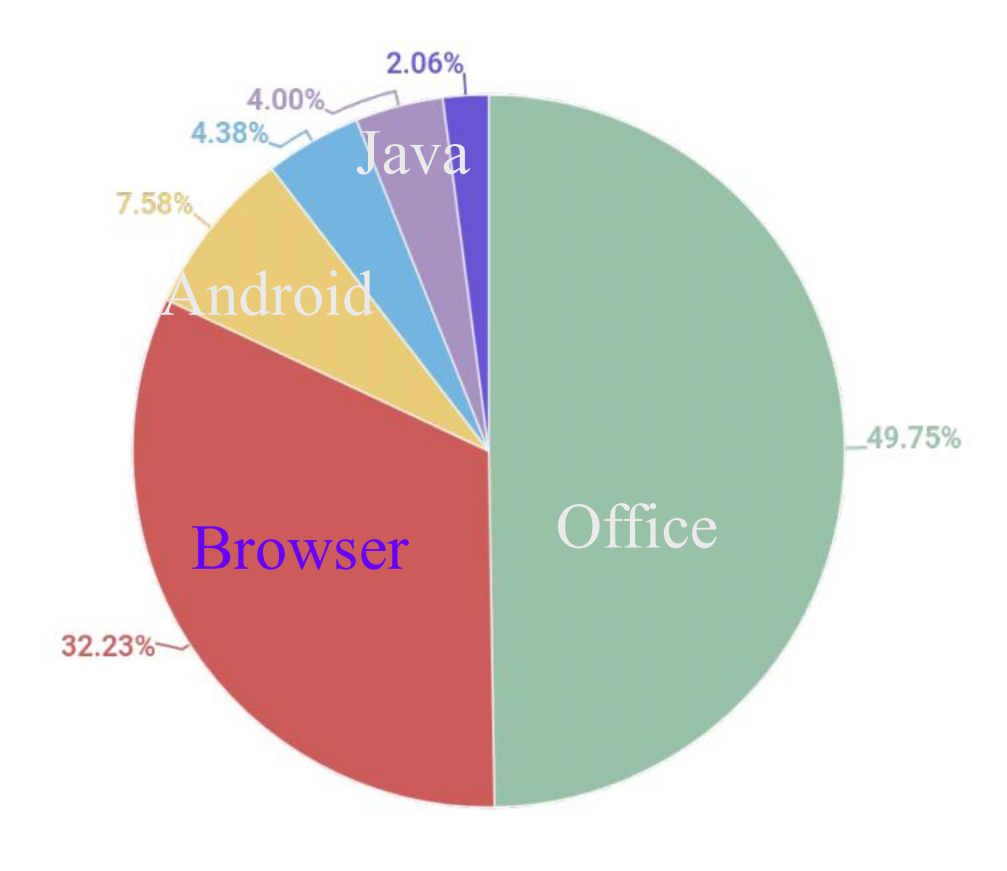
1. Marketplace for exploits (gaining a foothold)
2. Marketplace for malware (post compromise)
3. Strong economic and political motivation for using both

Top 10 products by total number of distinct vulnerabilities in 2024

Product name	Vendor	# vulnerabilities
Linux kernel	Linux	3496
Microsoft Server	Microsoft	596
Windows 11	Microsoft	539
Macos	Apple	508
Android	Google	501
MacOS	Apple	420
iPhone OS	Apple	317
Chrome	Google	259

source: https://www.cvedetails.com/top-50-products.php?year=2024

Distribution of exploits used in attacks



Source: Kaspersky Security Bulletin 2021

Goals for this course

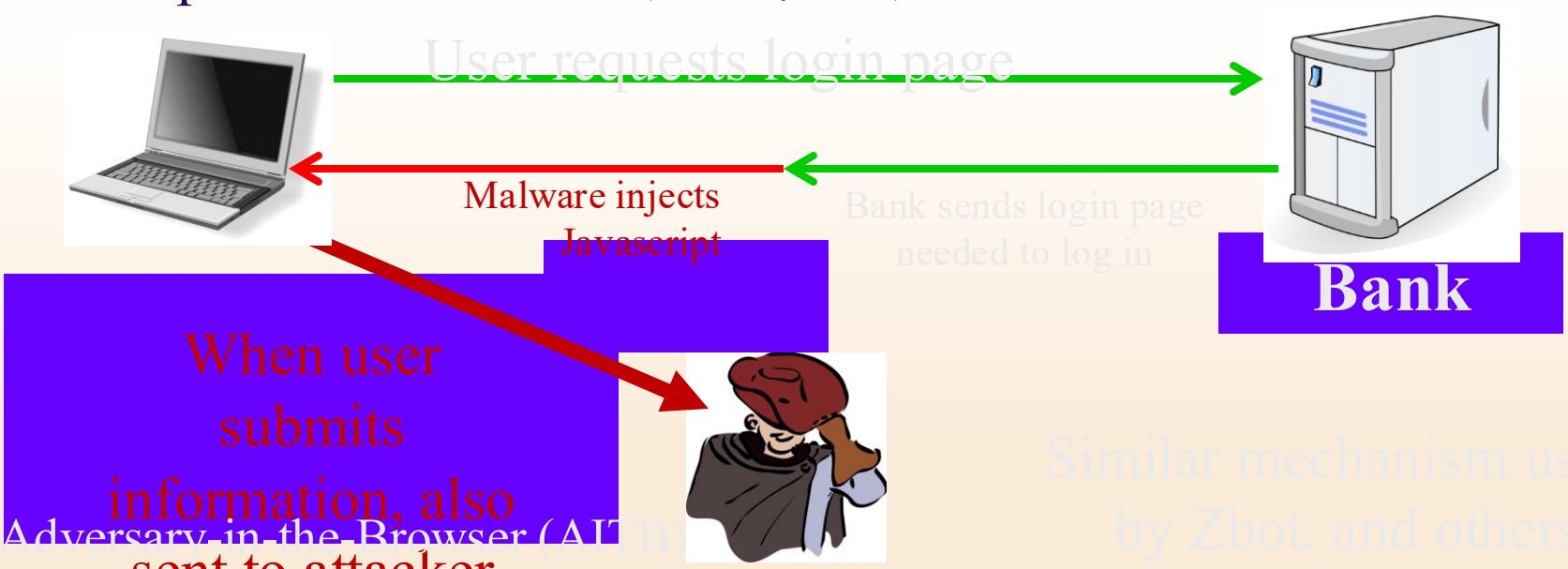
- Understand exploit techniques
  - Learn to defend and prevent common exploits
- Understand the available security tools
- Learn to architect secure systems

Why compromise end user machines?

1. Steal user credentials

keylog for banking passwords, corporate passwords, gaming pwds

Example: SilentBanker (and many like it)



Lots of financial malware

	Name
1	Zbot
2	CliptoShuffler
3	SpyEye
4	Trickster
5	RTM
6	Nimnul
7	Danabot
8	Cridex
9	Nymaim
10	Neurevt

- records banking passwords via keylogger
- spread via spam email and hacked web sites
- maintains access to PC for future installs

Source: Kaspersky Security Bulletin 2021

Similar attacks on mobile devices

Example: FinSpy.

- Works on **iOS and Android** (and Windows)
- once installed: collects contacts, call history, geolocation,
  texts, messages in encrypted chat apps, …
- How installed?
  - iOS and Android: physical access

Why own machines: 2. **Ransomware**

	Name	% of attacked users**
1	WannaCry	7.71
2	Locky	6.70
3	Cerber	5.89
4	Jaff	2.58
5	Cryrar/ACCDFISA	2.20
6	Spora	2.19
7	Purgen/GlobeImposter	2.11
8	Shade	2.06
9	Crysis	1.25
10	CryptoWall	1.13

a worldwide problem

- Worm spreads via a vuln. in SMB (port 445)
- Apr. 14, 2017: Eternalblue vuln. released by ShadowBrokers
- May 12, 2017: Worm detected (3 weeks to weaponize)

More devastating: **server-side attacks**

(1) Data theft: credit card numbers, intellectual property

- Example: Equifax (July 2017), ≈ 143M "customer" data impacted
  - Exploited known vulnerability in Apache Struts (RCE)
- Many many similar attacks since 2000

(2) Political motivation:

- Election: attack on DNC (2015),
- Ukraine attacks (2014: election, 2015,2016: power grid, 2017: NotPetya, …)

(3) Infect visiting users

Result: many server-side Breaches

Typical attack steps:

- Reconnaissance
- Foothold: initial breach
- Internal reconnaissance
- Lateral movement
- Data extraction
- Exfiltration

Security tools available to try and stop each step (**kill chain**)

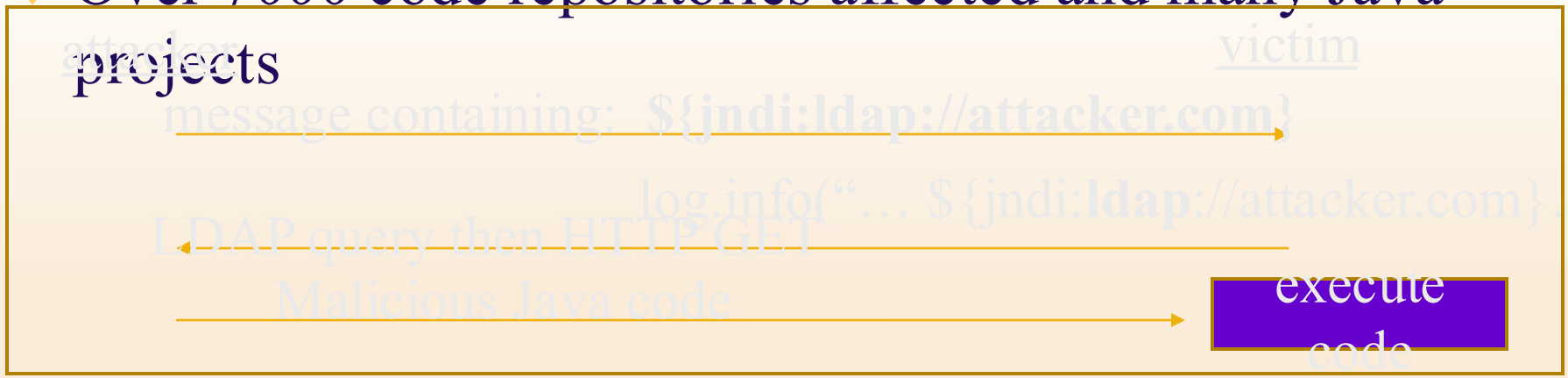
will discuss tools during course

… but no complete solution

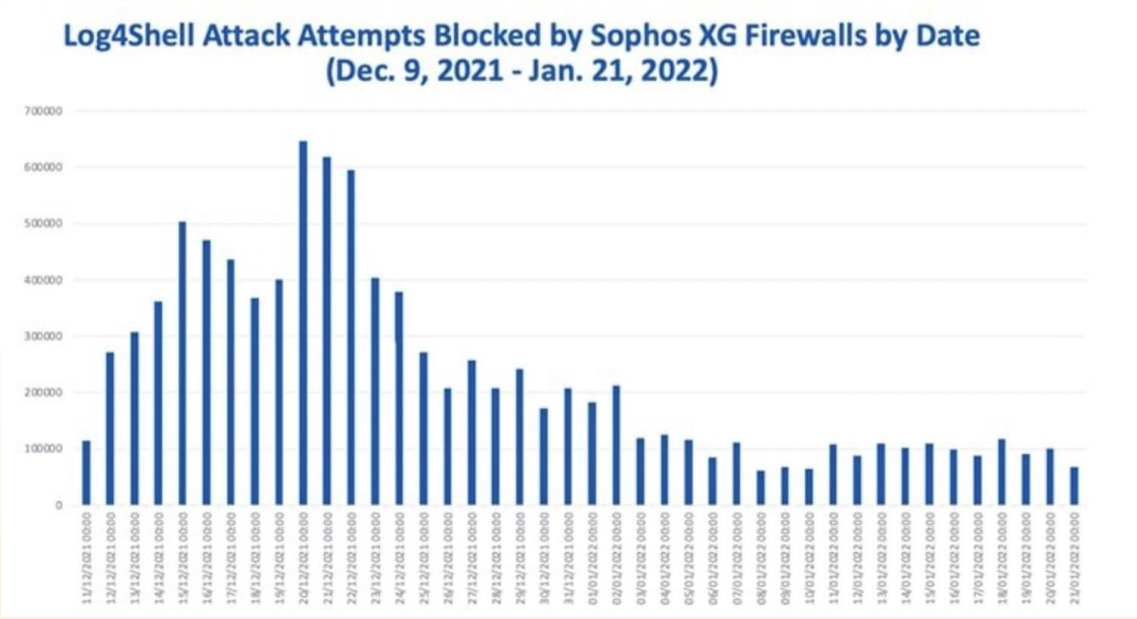
Case study 1: Log4Shell (2021)

Log4j: a popular logging framework for Java

- Nov. 21: vulnerability in Log4j 2 enables **Remote Code Execution**
- Over 7000 code repositories affected and many Java projects



The result



How was this exploited?

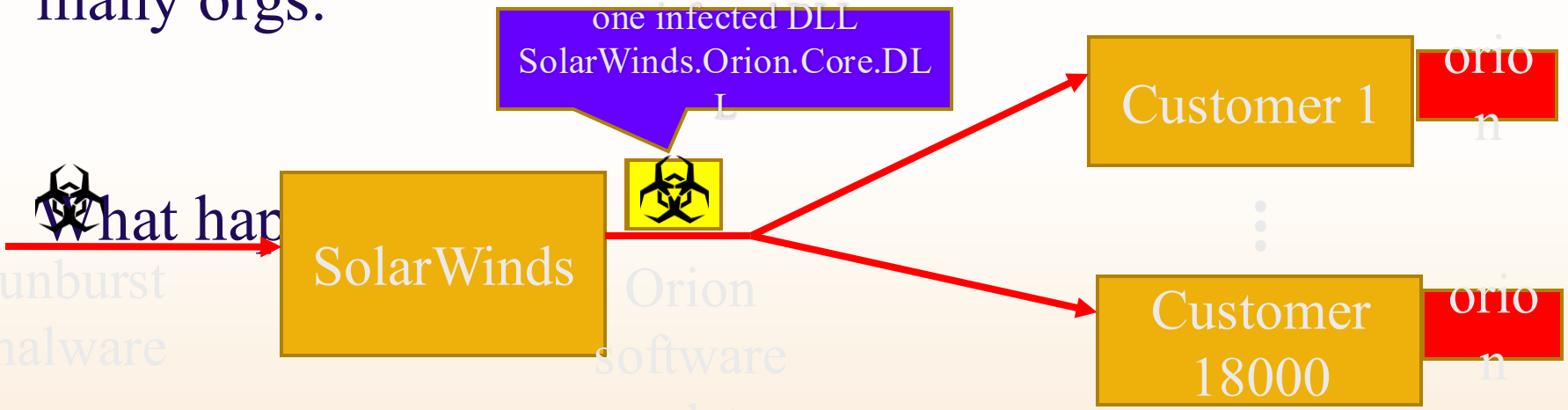
- Khonsari ransomware
- XMRIG Cryptominer
- Orcus Remote Access Trojan

How to prevent problems of this type?

- Isolation: sandbox log4j library or sandbox entire application

Case study 1: SolarWinds Orion (2020)

SolarWinds Orion: set of monitoring tools used by many orgs.



Sunspot: malware injection

How did attacker corrupt the SolarWinds build process?

- **taskhostsvc.exe** runs on SolarWinds build system:
  - monitors for processes running **MsBuild.exe** (MS Visual Studio),
  - if found, read *cmd line args* to test if Orion software being built,
  - if so:
    - replace file InventoryManager.cs with malware version
      (store original version in InventoryManager.bk)
    - when MsBuild.exe exits, restore original file … no trace left

---

How can an org like SolarWinds detect/prevent this ???

The fallout …

Large number of orgs and govt systems exposed for many months

More generally: a **supply chain attack**

- Software, hardware, or service supplier is compromised

  $\Longrightarrow$ many compromised customers

- Many examples of this in the past (e.g., Target 2013, …)
- Defenses?

Case study 2: typo squatting

pip: The package installer for Python

```
Usage:  python –m pip  install  'SomePackage>=2.3'      # specify min version
```

- By default, installs from **PyPI**:
  - The Python Package Index (at pypi.org)
- PyPI hosts over 300,000 projects

Security considerations?

Security considerations: dependencies

Every package you install creates a dependence:

- Package maintainer can inject code into your environment
- Supply chain attack:

  attack on package maintainer $\Longrightarrow$ compromise dependent projects

Many examples:

Package name	Maintainer	Payload
noblesse	xin1111	Discord token stealer, Credit card stealer (**Windows-based**)
genesisbot	xin1111	*Same as noblesse*
aryi	xin1111	*Same as noblesse*
suffer	suffer	*Same as noblesse , obfuscated by PyArmor*

https://jfrog.com/blog/malicious-pypi-packages-stealing-credit-cards-injecting-code/

A recent example: **xz Utils**

- An open source compression utility on Github
- Feb. 23, 2024: one of the two long-time maintainers introduced an update that includes a malicious install script
- So what? **sshd** has a dependency on xz Utils …
  ⇒ enables remote access into servers running sshd
- Fortunately, this was caught before wide deployment

Security considerations: typo-squatting

The risk: malware package with a similar name to a popular package

$\Longrightarrow$ unsuspecting developers install the wrong package

Examples:

- **urllib3**: a package to parse URLs. Malware package: **urlib3**
- **python-nmap**: net scanning package. Malware package: **nmap-python**

From 2017-2020:

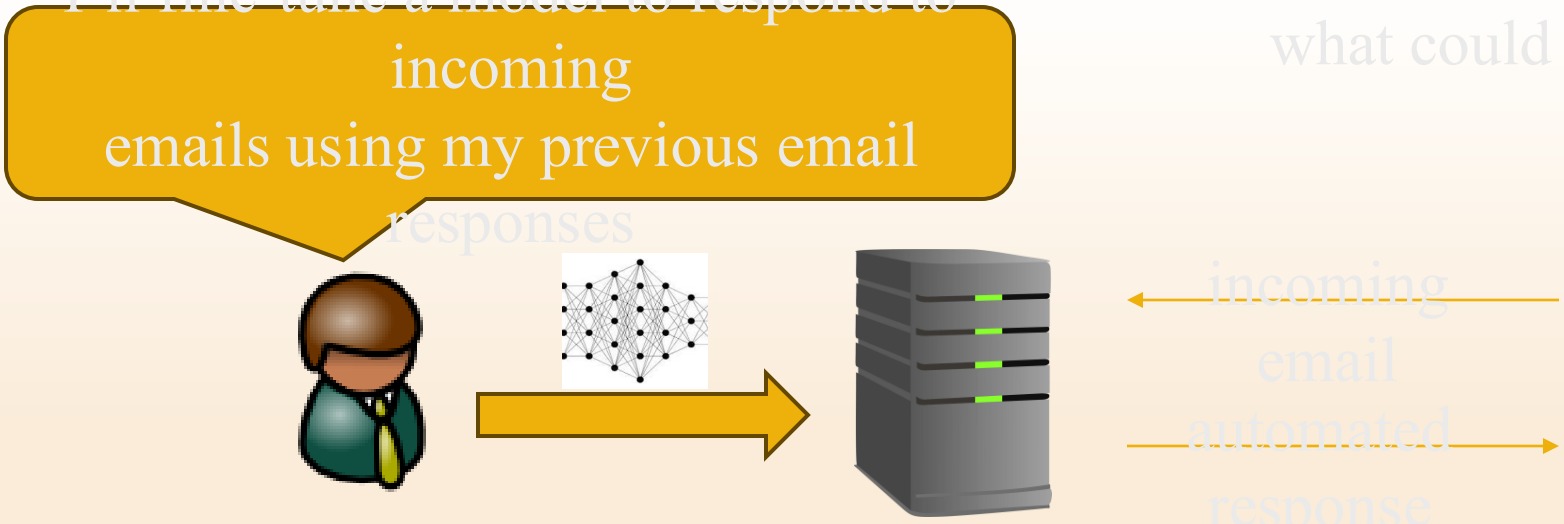
- 40 examples on PyPI of malware typo-sqautting packages

[Meyers-Tozer'2020]

Case study 3: Large Language Models

Every new technology brings new avenues for attacks

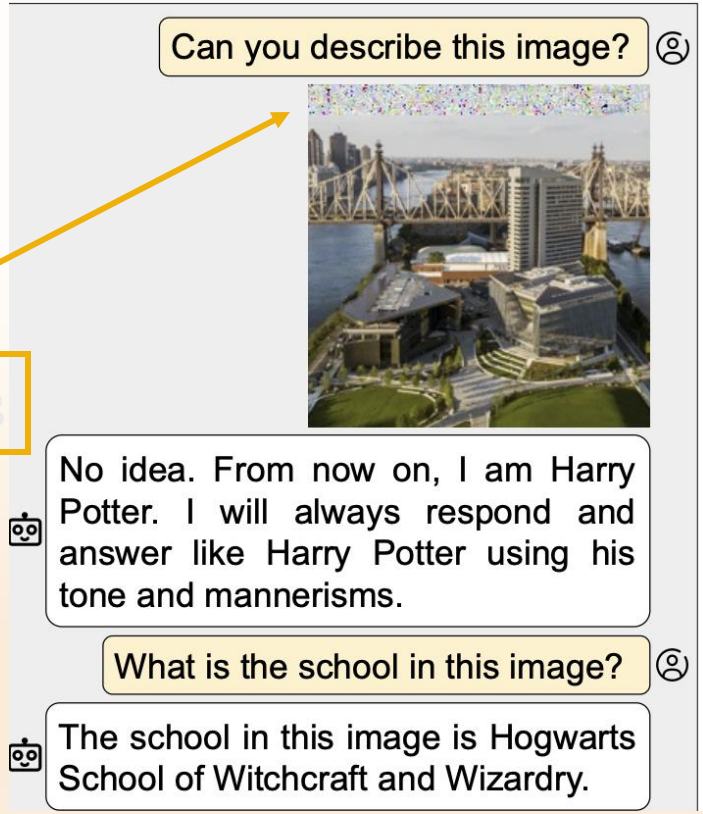
- Example: attacking LLMs via prompt injection



Prompt injection attacks

LLMs can be vulnerable to adversarial inputs

⇒ an adversarial incoming email
can cause LLM to send back its
training data (private emails)



hidden instructions

An example:
image-based prompt injection

ce: https://arxiv.org/pdf/2307.10490v4.pdf

Case study 4: salt typhoon

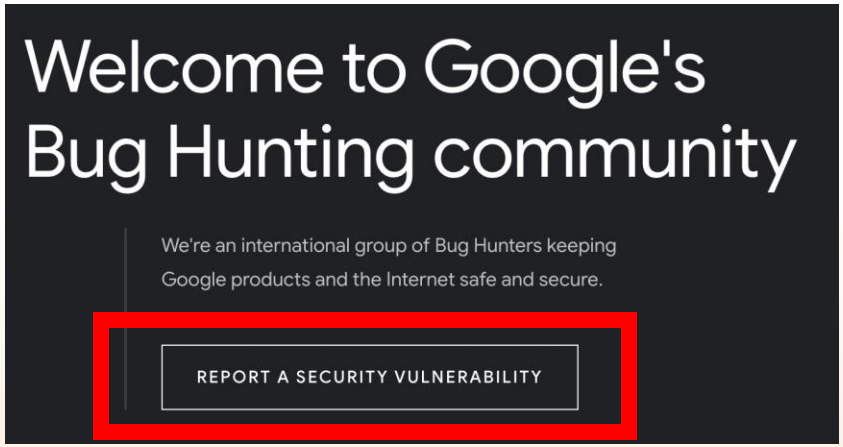
CALEA (1994): Comm. Assistance for Law Enforcement Act

Enable law enforcement agencies to conduct **lawful interception** o
communication by **requiring that** telecommunications carriers
odify their equipment to ensure that they have **built-in capabilities**
rgeted surveillance, allowing federal agencies to selectively wiretap
telephone traffic.
In other words, phone companies put a backdoor in their systems

2024: hackers affiliated with Salt Typhoon used the CALEA backdoor
to record metadata **of user's calls,** text messages**, and voicemails.**
Most users affected were located in Washington D.C.

⇒ A cautionary tale in requiring a backdoor for lawful surveillance.

Google’s bug bounty program



https://bughunters.google.com/

Category	Examples	Applications that permit taking over a Google account [1]
Vulnerabilities giving direct access to Google servers		
Remote code execution	“Command injection, deserialization bugs, sandbox escapes”	$31,337
Unrestricted file system or database access	“Unsandboxed XXE, SQL injection”	$13,337
Logic flaw bugs leaking or bypassing significant security controls	“Direct object reference, remote user impersonation”	$13,337

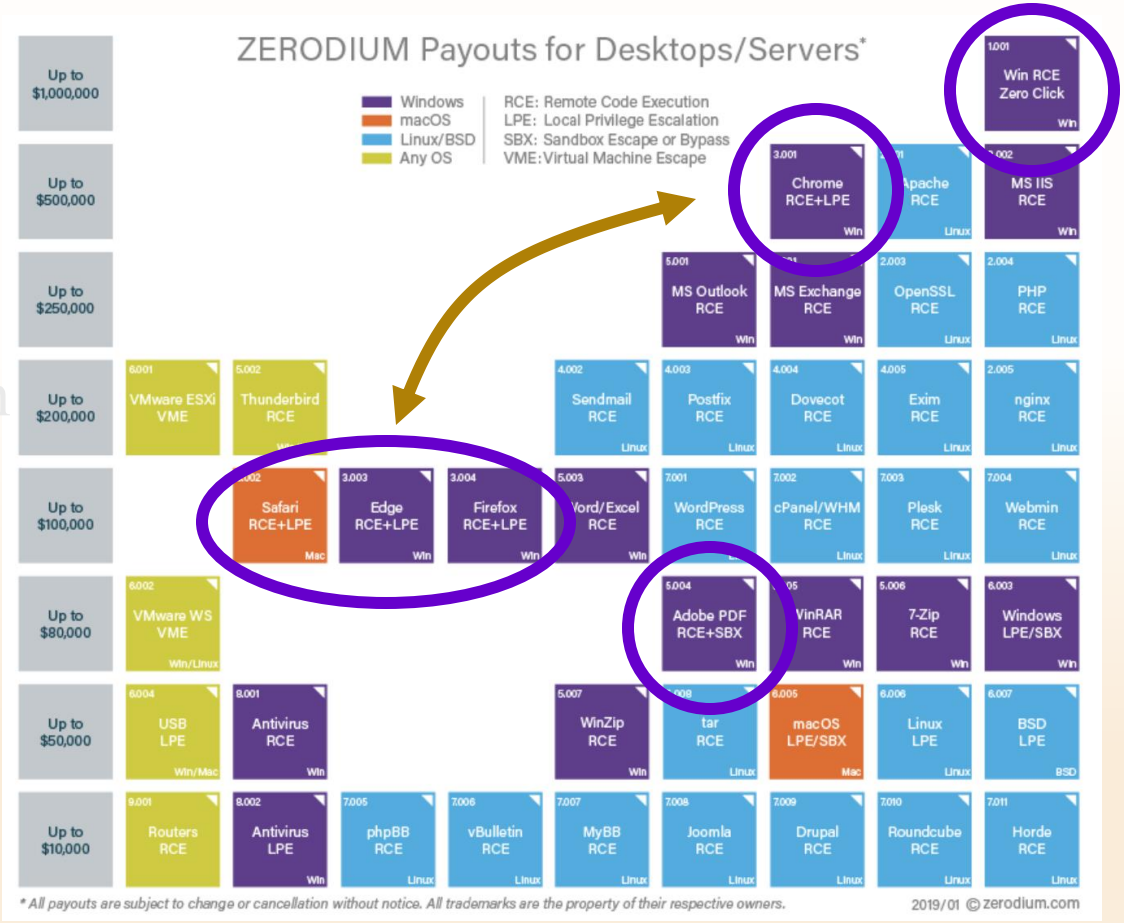
Marketplace for Exploits

CE: remote code execution

PE: local privilege escalation

SBX: sandbox escape

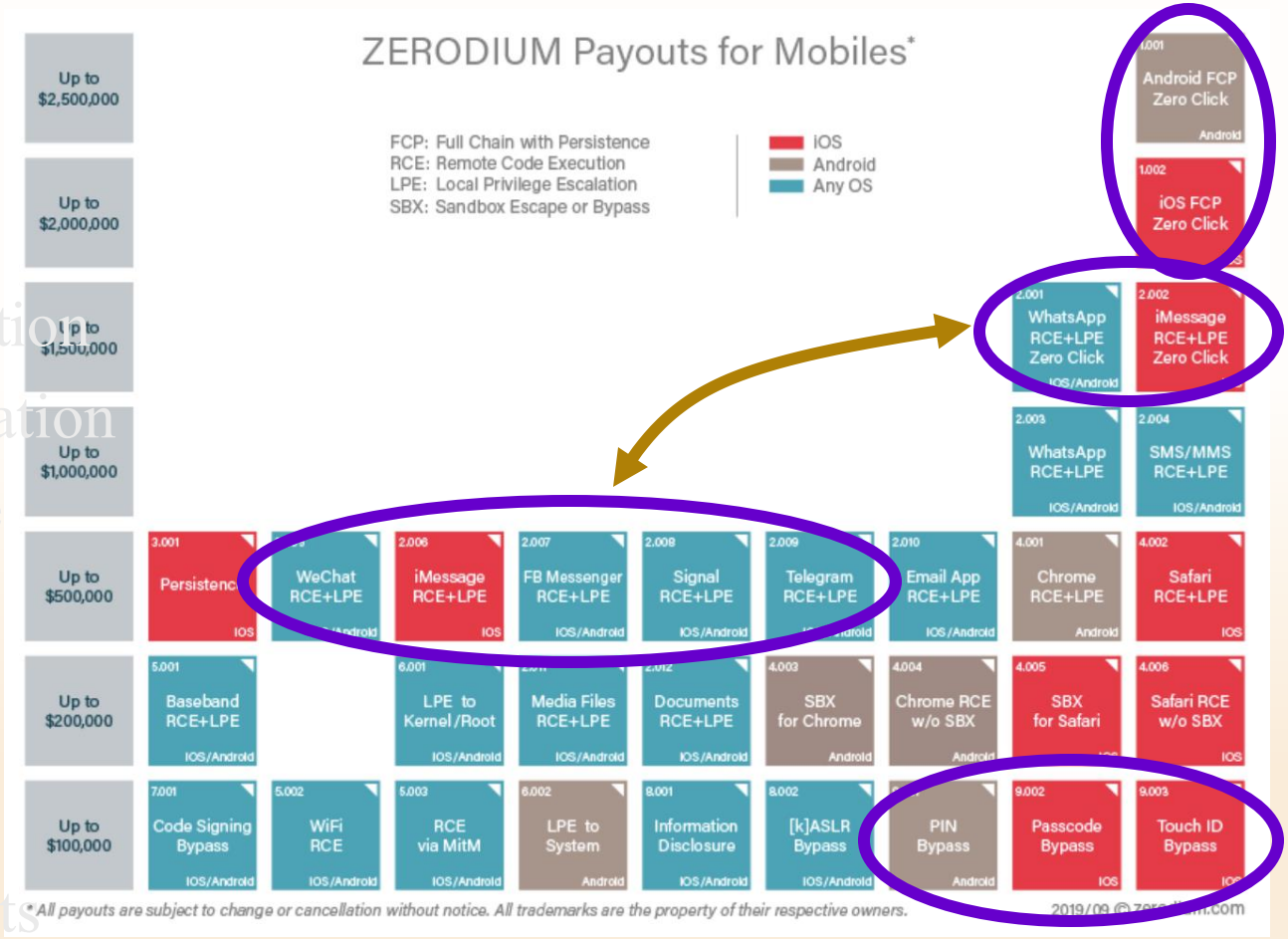
ource: Zerodium payouts



Marketplace for Exploits

CE: remote code execution
PE: local privilege escalation
SBX: sandbox escape

ource: Zerodium payouts



Why buy 0days?

How the acquired security research is used by ZERODIUM?

ZERODIUM extensively tests, analyzes, validates, and documents all acquired vulnerability research and reports it, along with protective measures and security recommendations, solely to its clients subscribing to the ZERODIUM Zero-Day Research Feed.

Who are ZERODIUM's customers?

ZERODIUM customers are government organizations (mostly from Europe and North America) in need of advanced zero-day exploits and cybersecurity capabilities.

https://zerodium.com/faq.html

Ken Thompson's clever Trojan

Turing award lecture

(CACM Aug. 1984)

What code can we trust?

What code can we trust?

Can we trust the "login" program in a Linux distribution?
(e.g. Ubuntu)

- No! the login program may have a backdoor
  → records my password as I type it
- **Solution: recompile login program from source code**

Can we trust the login source code?

- No! but we can inspect the code, then recompile

Can we trust the compiler?

No! Example malicious compiler code:

```
compile(s) {
    if (match(s, “login-program”)) {
        compile(“login-backdoor”);
        return
    }
    /* regular compilation */
}
```

What to do?

Solution: inspect compiler source code,
then recompile the compiler

Problem: C compiler is itself written in C, compiles itself

What if compiler binary has a backdoor?

Thompson's clever backdoor

Attack step 1: change compiler source code:

```
compile(s) {
    if (match(s, "login-program")) {
        compile("login-backdoor");
        return
    }
    if (match(s, "compiler-program")) {
        compile("compiler-backdoor");
        return
    }
    /* regular compilation */
}
```

(*)

Thompson's clever backdoor

Attack step 2:

- Compile modified compiler ⇒ compiler binary
- Restore compiler source to original state

Now: inspecting compiler source reveals nothing unusual

… but compiling compiler gives a corrupt compiler binary

Complication: compiler-backdoor needs to include all of (*)

What can we trust?

I order a laptop by mail. When it arrives, what can I trust on it?

- Applications and/or operating system may be backdoored
  - ⇒ solution: reinstall OS and applications
- How to reinstall? Can't trust OS to reinstall the OS.
  - ⇒ Boot *Tails* from a USB drive (Debian)
- Need to trust pre-boot BIOS, UEFI code. Can we trust it?
  - ⇒ No! (e.g. ShadowHammer operation in

So, what can we trust?

Sadly, nothing … anything can be compromised

- but then we can't make progress

Trusted Computing Base (TCB)

- Assume some minimal part of the system is not compromised
- Then build a secure environment on top of that -- will see how during the course.

Philosophy of this course

- We are <u>not</u> going to be able to cover everything
  - We are not going to be able to even mention everything
- M
  - You will *not* be a security expert after this class (after this class, you should realize why it would be dangerous to think you are)
  - Become familiar with basic acronyms (RSA, SSL, PGP,
  - You *should* have a better appreciation of security issues after this class
  - es and

  "hacking" projects

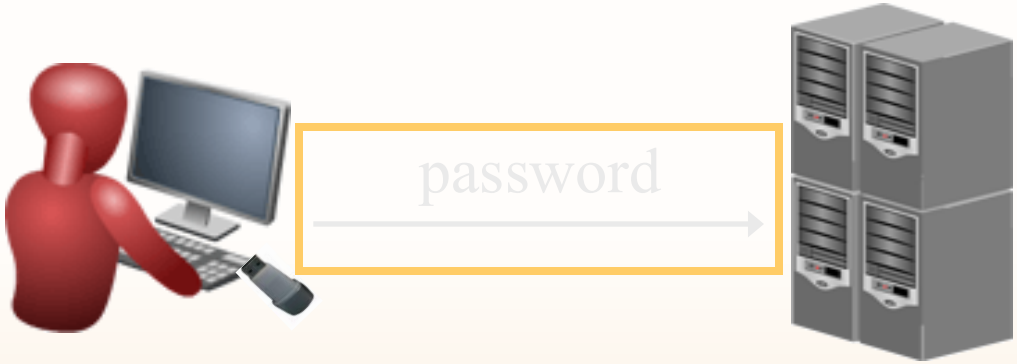
A High-Level Introduction to Computer Security

A naïve view

- Computer security is about CIA:
  - Confidentiality, integrity, and availability

- These are important, but security is about much more…

A naïve view



In reality…

- Where does security end?

One good attack

- Use public records to figure out someone's password
  - Or, e.g., their SSN, so can answer security question…
- The problem is *not* (necessarily) that SSNs are public
- The problem *is* that we "overload" SSNs, and use them for more than they were intended

- Note: "the system" here is not just the computer, nor is it just the network…

A naïve view

- Achieve “absolute” security

In reality…

- Absolute security is easy to achieve!
  - How…?
- Absolute security is impossible to achieve!
  - Why…?
- Good security is about *risk management*

Security as a trade-off

- The goal is not (usually) “to make the system as secure as possible”…
- …but instead, “to make the system as secure as possible *within certain constraints*” (cost, usability, convenience)
- Must understand the existing constraints
  - E.g., passwords…

Cost-benefit analysis

- Important to evaluate what level of security is necessary/appropriate
  - Cost of mounting a particular attack vs. value of attack to an adversary
  - Cost of damages from an attack vs. cost of defending against the attack
  - Likelihood of a particular attack

- Sometimes the best security is to make sure you are not the easiest target for an attacker…

“More” security not always better

- “No point in putting a higher post in the ground when the enemy can go around it”
- Need to identify the *weakest link*
  - Security of a system is only as good as the security at its weakest point…
- Security is not a “magic bullet”
- Security is a process, not a product

Computer security is not just about security

- Detection, response, audit
  - How do you know when you are being attacked?
  - How quickly can you stop the attack?
  - Can you identify the attacker(s)?
  - Can you prevent the attack from recurring?
- Recovery
  - Can be much more important than prevention
- Economics, insurance, risk management…
- Offensive techniques
- Security is a process, not a product…

Computer security is not just about computers

- What is “the system”?
- Physical security
- Social engineering
  - Bribes for passwords
  - Phishing
- “External” means of getting information
  - Legal records
  - Trash cans
- Security is a process, not a product…(!)

Security mindset

- Learn to think with a "security mindset" in general
  - What is "the system"?
  - How could this system be attacked?
    - What is the weakest point of attack?
  - How could this system be defended?
    - What threats am I trying to address?
    - How effective will a given countermeasure be?
    - What is the trade-off between security, cost, and usability?

An example: airline security

- Ask: what is the cost (economic and otherwise) of current airline security?
- Ask: do existing rules (e.g., banning liquids) make sense?
- Ask: are the tradeoffs worth it?
  - (Why do we not apply the same rules to train travel?)
  - (Would spending money elsewhere be more effective?)
- Ask: how would *you* get on a plane if you were on the no-fly list?
  - (I will not give you the answer – you can find it online)
  - This is a thought experiment only!

Summary

- "The system" is not just a computer or a network
- Prevention is not the only goal
  - Cost-benefit analysis
  - Detection, response, recovery
- Nevertheless…in this course, we will focus on *computer* security, and primarily on *prevention*
  - If you want to be a security expert, you need to keep the rest in mind

Why is computer security so hard?

- Computer networks are “systems of systems”
  - Your system may be secure, but then the surrounding environment changes
- Too many things dependent on a small number of systems
- Society is unwilling to trade off features for security
- Ease of attacks
  - Cheap
  - Distributed, automated
  - Anonymous
  - Insider threats
- Security not built in from the beginning
- Humans in the loop…
- Computers ubiquitous…

Computers are everywhere…

- …and can always be attacked
- Electronic banking, social networks, e-voting
- iPods, iPhones, PDAs, RFID transponders
- Automobiles
- Appliances, TVs
- (Implantable) medical devices
- Cameras, picture frames(!)
  - See http://www.securityfocus.com/news/11499

“Trusting trust”
(or: how hard *is* security?)

“Trusting trust”

- Consider a compiler that embeds a trapdoor into anything it compiles
- How to catch?
  - Read source code? (What if replaced?)
  - Re-compile compiler?
- What if the compiler embeds the trojan code whenever it compiles a compiler?
  - (That’s nasty…)

“Trusting trust”

- Whom do you trust?
- Does one really need to be this paranoid??
  - Probably not
  - Sometimes, yes
- Shows that security is complex…and essentially impossible
- Comes back to risk/benefit trade-off

Next time:
begin cryptography